

ARTI 403 — Image Processing

Lab 4 — Intensity Transformations and Filtering (Spatial Domain)

Student name	Ahmed Albuainain
Student ID	2240006128
Section	7MA1
College	Computer Science and Information Technology
Department	Computer Engineering
University	Imam Abdulrahman Bin Faisal University
Date	5 October 2026

1. Objective

Outcome #2: Write a program that implements fundamental image processing algorithms.

This lab covers two families of intensity operation. **Thresholding** reduces an image to two values by comparing every pixel against a single number, and is the simplest form of segmentation. **Histogram processing** reshapes the distribution of intensities to improve contrast — by stretching it, by flattening it, or by matching it to another image.

2. Tools

Component	Detail
Python	3 (Anaconda)
IDE	Jupyter Notebook (via Visual Studio Code)
Libraries	OpenCV, NumPy, Pillow, scikit-image, Matplotlib
Images	Parrot.png (on disk); moon, rocket, chelsea, coins from skimage.data

3. Task 1 — Thresholding

Thresholding converts a grayscale image into a binary one. Pixels above the threshold become foreground (white, 255) and pixels below become background (black, 0), which simplifies the image to two categories and makes segmentation and feature extraction tractable.

3.1 Fixed thresholds

The five threshold values from the manual were applied to the same image. The proportion of white pixels makes the effect precise in a way the images alone do not.

Threshold	White pixels	% of image
0	239,998	100.0%
50	189,667	79.0%
100	122,212	50.9%

150	48,317	20.1%
200	13,589	5.7%

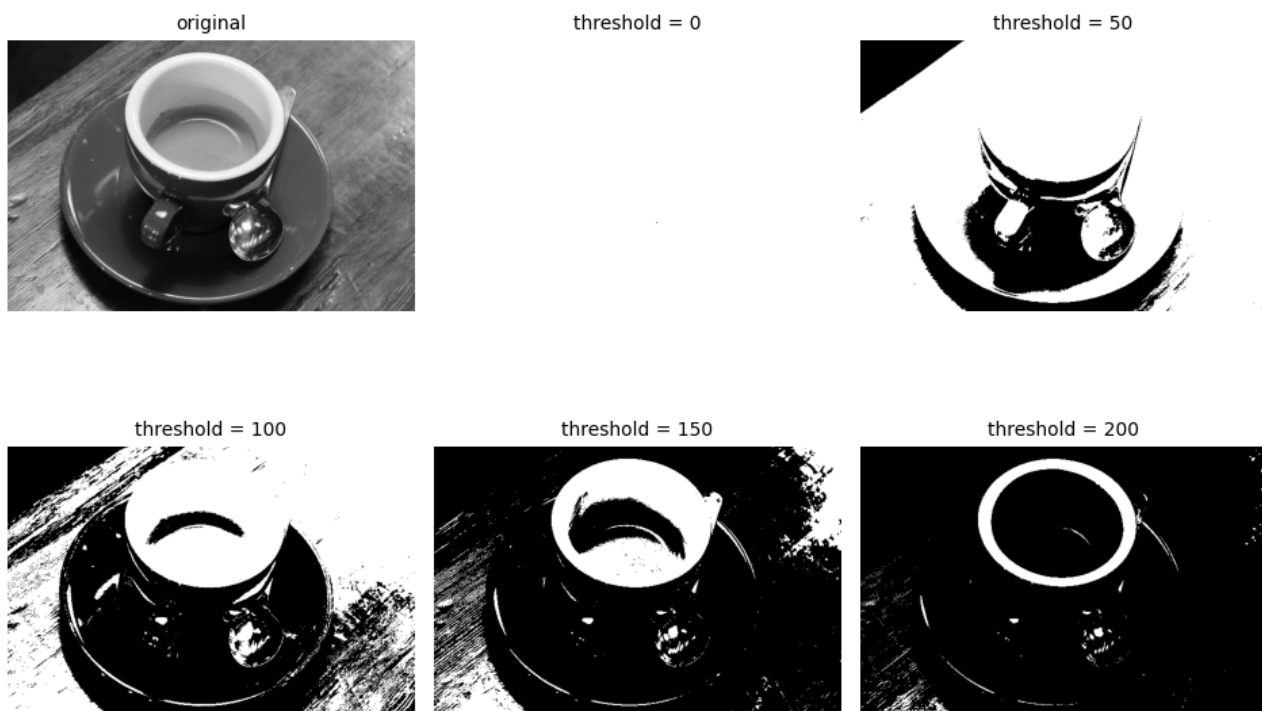


Figure 1 — the original image and the result of thresholding at 0, 50, 100, 150 and 200.

Two observations follow. First, a threshold of **0** is **not a no-op** and is not quite "everything white" either: `THRESH_BINARY` sets a pixel to 255 when it is *strictly greater* than the threshold, so every pixel except the pure-black ones turns white. This image contains exactly 2 pixels of value 0, and the thresholded result contains exactly that many black pixels — the two were verified to match.

Second, the useful threshold depends entirely on the image. As the value rises, more of the image falls below it and turns black, and there is no setting that is correct in general. That is the difficulty the manual describes, and the reason the automatic methods below exist.

3.2 The other thresholding types

`cv2.threshold` offers five modes. Two of them do not produce binary images at all: `TRUNC` caps the bright values at the threshold and leaves the rest untouched, and `TOZERO` zeroes the dark values and leaves the bright ones at their original intensity.

Type	Behaviour
<code>THRESH_BINARY</code>	above the threshold becomes 255, below becomes 0
<code>THRESH_BINARY_INV</code>	the inverse of the above
<code>THRESH_TRUNC</code>	above becomes the threshold, below unchanged
<code>THRESH_TOZERO</code>	above unchanged, below becomes 0
<code>THRESH_TOZERO_INV</code>	above becomes 0, below unchanged



Figure 2 — the five thresholding types at a threshold of 120.

3.3 Choosing the threshold automatically — Otsu's method

Otsu's method examines the histogram and selects the value that best separates the pixels into two groups, by maximising the variance between them. No manual tuning is required. It chose 104 for the lab image and 107 for the coins image, which is the classic test case for segmentation.

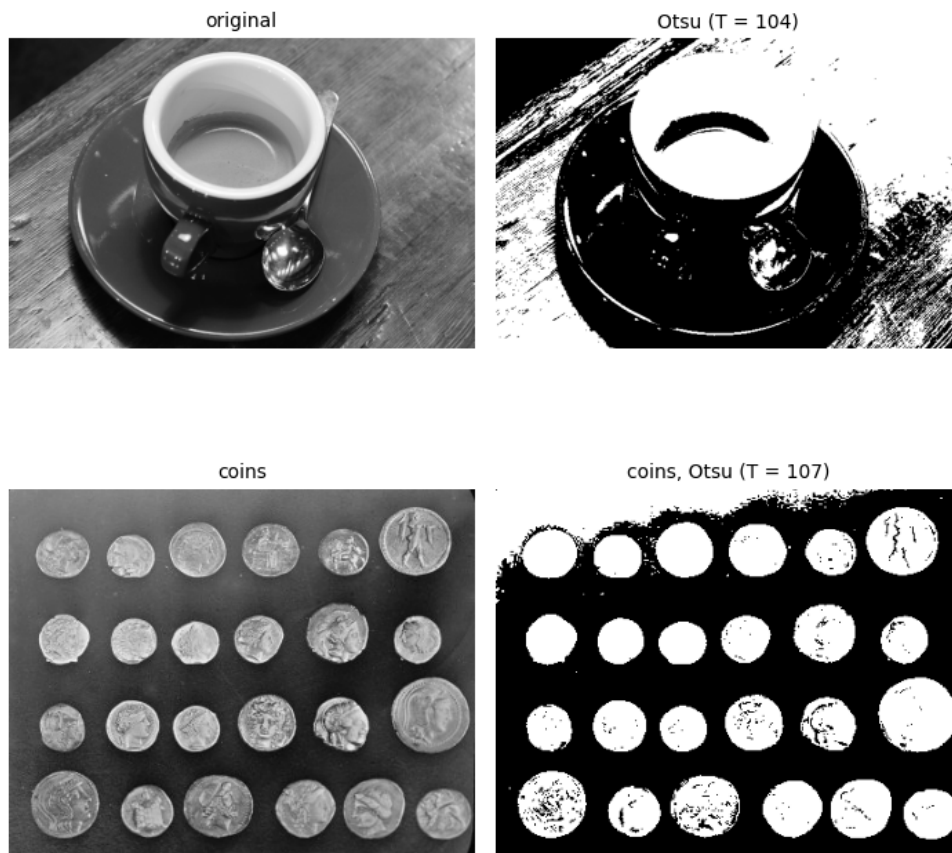


Figure 3 — Otsu thresholding applied to both images.

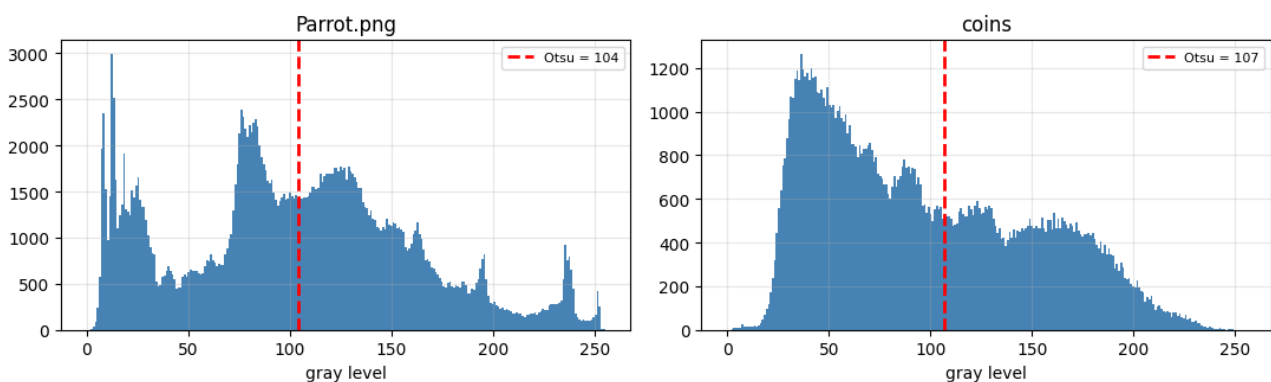


Figure 4 — where Otsu placed the threshold on each histogram.

The comparison is instructive. The coins histogram has two clearly separated peaks and Otsu lands in the valley between them, producing a clean segmentation. The lab image has a more continuous histogram with no such valley, so the same method gives a reasonable but far less decisive result. Otsu is only as good as the assumption that the image really does contain two distinct populations of pixels.

3.4 Adaptive thresholding

When illumination is uneven, no single global threshold can work: it will inevitably be too high for the dark side of the image and too low for the bright side. To demonstrate this, the image was multiplied by a

left-to-right brightness ramp and then thresholded both ways.

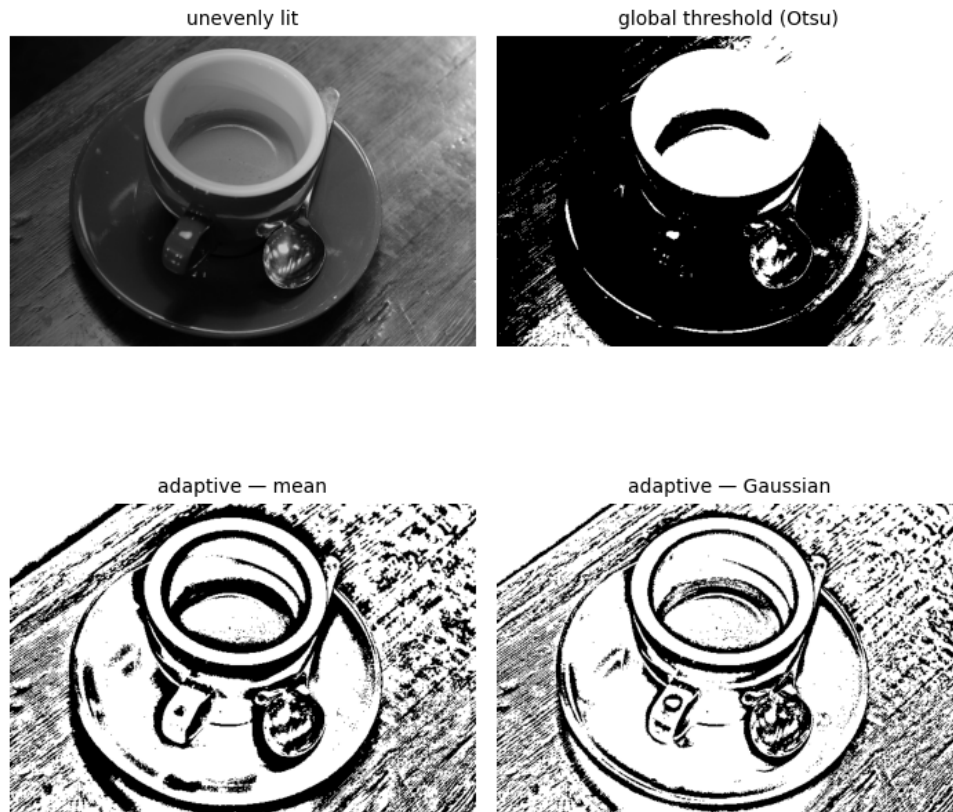


Figure 5 — an unevenly lit image thresholded globally and adaptively.

The global threshold fails exactly as predicted — the darkened left side falls below it almost entirely — while the adaptive versions compute a separate threshold for each neighbourhood and preserve detail across the whole frame.

4. Task 2 — Histogram processing

A histogram counts how many pixels hold each intensity value. It says nothing about where those pixels are, but it describes the tonal character of an image completely, and the contrast operations below are best understood as different ways of reshaping it.

4.1 The moon image

The image looks flat and washed out, and the histogram shows why. Although the values do reach 0 and 255, 98% of the pixels lie between 78 and 129 — a span of only 51 of the 256 available levels. The full range is technically used, but only by a handful of outliers.

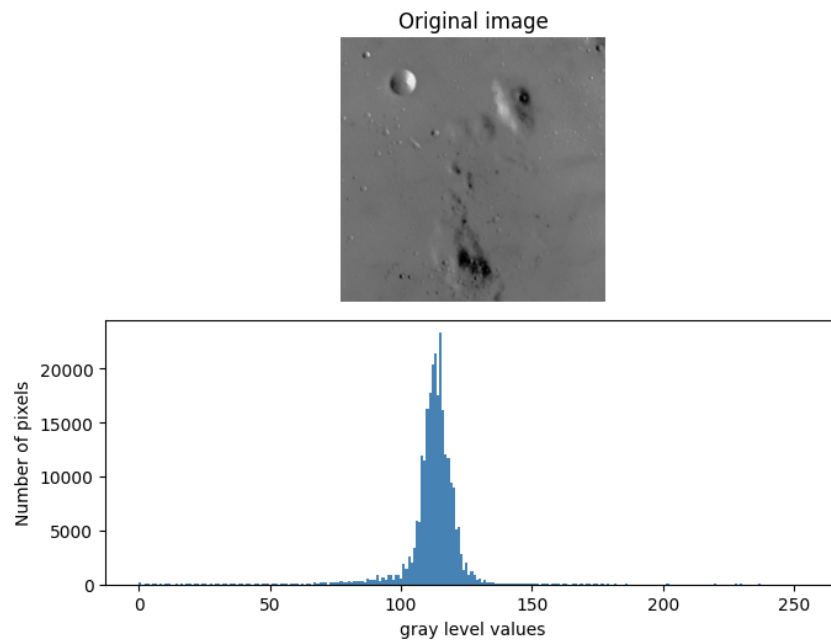


Figure 6 — the original moon image and its histogram.

4.2 Contrast stretching — 2nd to 98th percentiles

Contrast stretching maps a chosen input range onto the full output range. Using percentiles rather than the minimum and maximum prevents a few extreme outliers from dictating the result. The input range 78–129 was expanded to 0–255, raising the standard deviation from 13.33 to 42.27.

Contrast stretch (2nd - 98th percentiles)

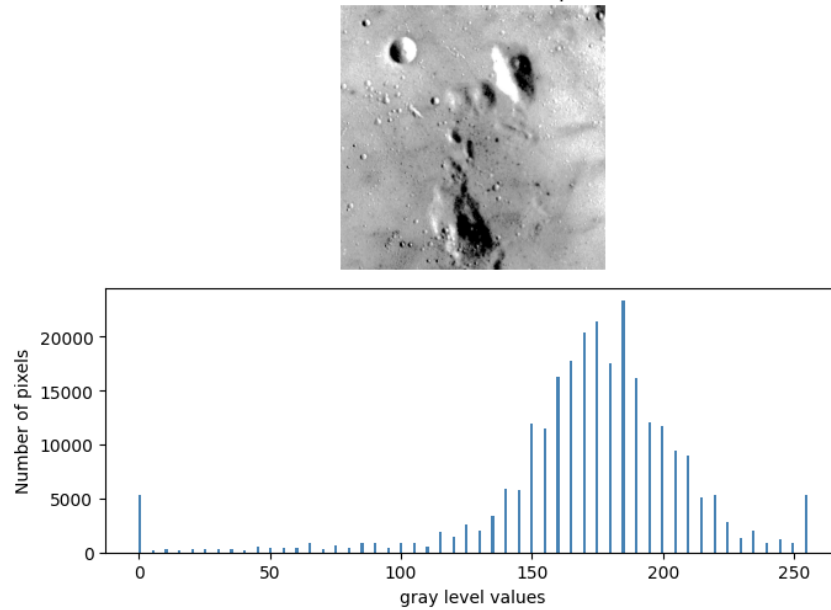


Figure 7 — contrast stretch to the 2nd–98th percentiles.

The shape of the distribution is unchanged; it has only been spread out. That is the defining property of contrast stretching and what distinguishes it from equalisation. Pixels outside the chosen range are **clipped** — everything below the lower bound becomes 0 and everything above the upper becomes 255 — so detail in the tails is lost.

5. Assessment Task 1 — Rescale to the 3rd and 80th percentiles

The same operation with a narrower input range, 87–118. The upper bound is far more aggressive than before.

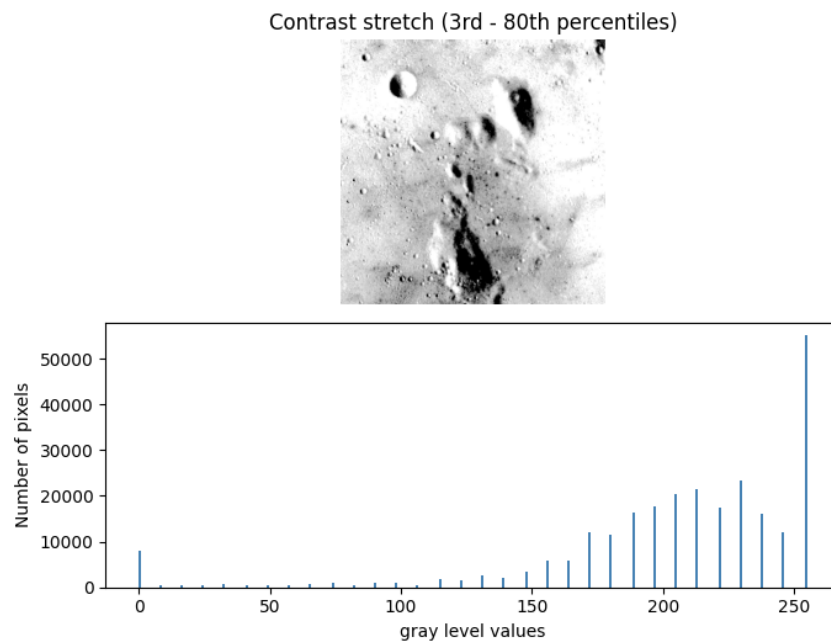


Figure 8 — contrast stretch to the 3rd–80th percentiles.

Version	Input range	Std. deviation	Clipped to white
Original	0 – 255	13.33	0.0%
2nd – 98th percentiles	78 – 129	42.27	1.9%
3rd – 80th percentiles	87 – 118	55.46	16.5%

The 3rd–80th version has visibly higher contrast in the mid-tones, and the standard deviation confirms it. The cost is that about one pixel in six is now saturated to pure white with no detail left in it, against roughly one in fifty for the 2nd–98th version.

The clipped fraction is below the 20% that an 80th-percentile cut-off might suggest. The reason is that the image is discrete and many pixels share the value at the cut-off, while only those strictly above it are clipped. This is the central trade-off of contrast stretching: every bit of contrast gained in the middle of the range is paid for with detail lost at the ends, and which setting is preferable depends on where the information of interest happens to sit.

6. Assessment Task 2 — Histogram equalisation

`exposure.equalize_hist` does something fundamentally different from contrast stretching. Rather than spreading the existing distribution out linearly, it redistributes the values so that every intensity level holds roughly the same number of pixels, using the cumulative distribution function of the image as the transformation curve.

6.1 An implementation detail that matters

`equalize_hist` returns a **float image with values between 0 and 1** — measured range 0.0009 to 1.00 — not `uint8` values between 0 and 255. Plotting its histogram with `range=(0, 255)`, as the manual's plotting code does for the original image, puts 99.6% of the pixels into the very first bin and produces an apparently empty chart. The display helper used in the notebook reads the range from the data instead, so it works correctly for both types.

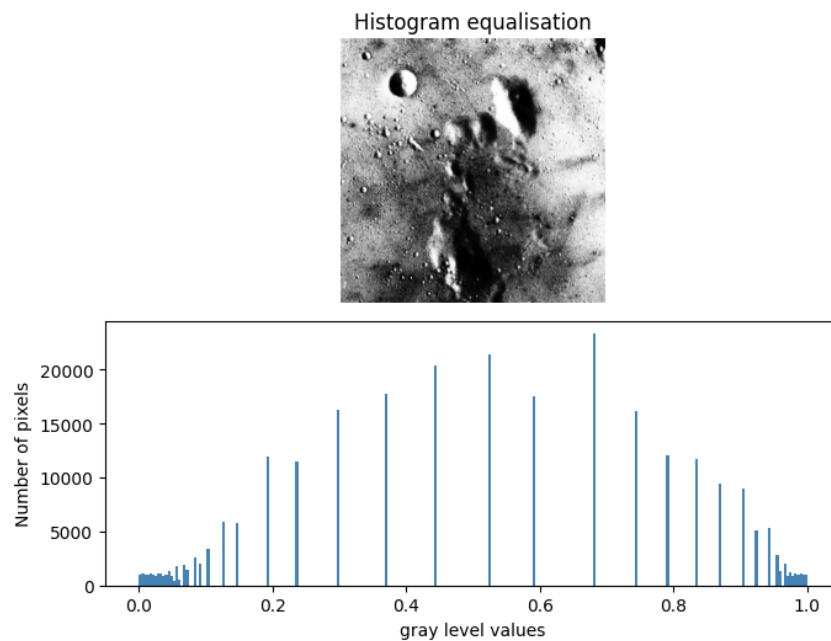


Figure 9 — the equalised image and its histogram.

6.2 Why the histogram is not perfectly flat

The histogram is spread across the whole range but still spiky. This is a property of equalising a **discrete** image, not an error. All the pixels sharing one input value are mapped to the same output value and cannot be split apart, so a tall spike in the input remains a tall spike in the output, merely relocated. Perfect flatness is achievable only for continuous data. The cumulative histogram is the right place to look: for a perfectly equalised image it is a straight diagonal.

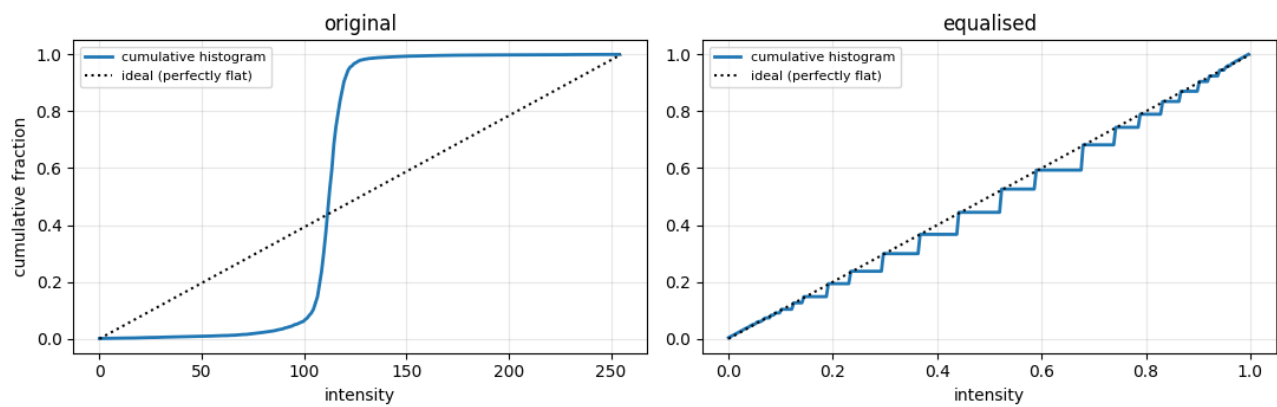


Figure 10 — cumulative histograms before and after equalisation, against the ideal diagonal.

6.3 Comparing the contrast methods

Adaptive equalisation (CLAHE) is included for completeness: it equalises small tiles rather than the whole image, which preserves local detail without over-amplifying global contrast.

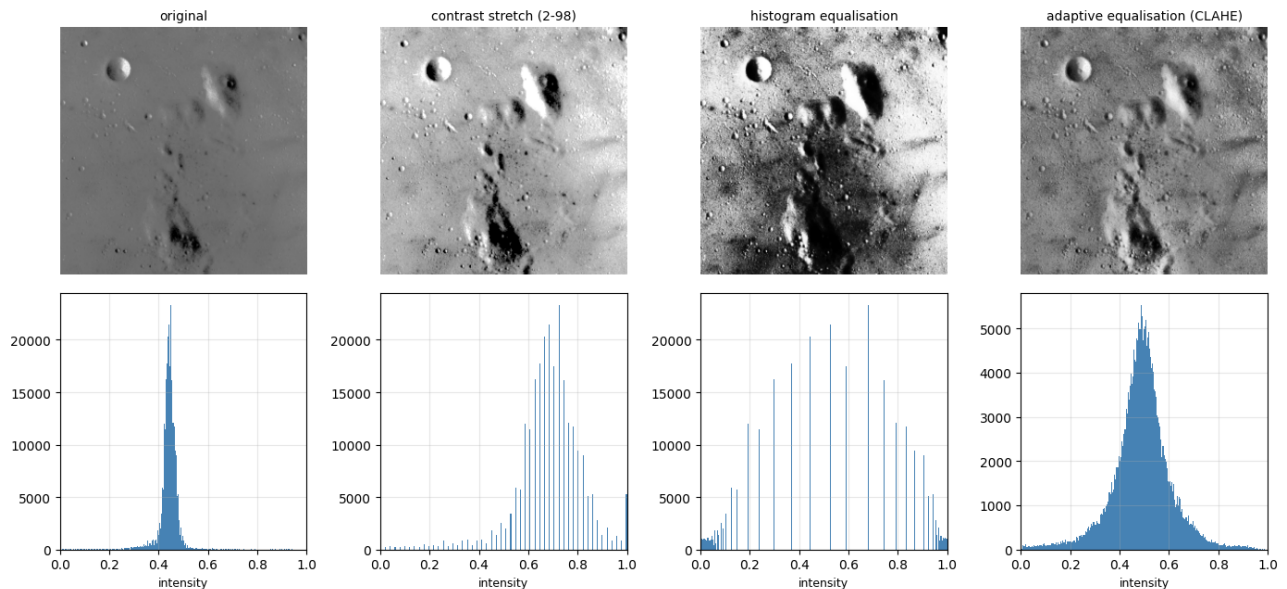


Figure 11 — the four versions of the moon image with their histograms.

Method	Standard deviation
original	0.0523
contrast stretch (2-98)	0.1658
histogram equalisation	0.2896
adaptive equalisation (CLAHE)	0.1195

7. Assessment Task 3 — Histogram matching

Histogram matching is the general case of the two previous operations. Instead of stretching a distribution or flattening it, it reshapes the histogram of one image so that it follows the histogram of a **reference** image. The input keeps its own content and takes on the reference's tonal character.

As the task specifies, the **rocket** image is the reference and **Chelsea** is the input. Both are colour images, so the matching runs channel by channel. Note that current scikit-image uses the `channel_axis` argument for this; the `multichannel` argument found in older tutorials has been removed.



Figure 12 — source, reference and matched image.

The matched image keeps the cat — its shapes and edges are untouched — but has taken on the cooler, more contrasted tone of the rocket photograph. Nothing was transferred between the two images except the distribution of values.

7.1 Verifying the match

The visual result is suggestive; the cumulative histograms are the proof. If the matching worked, the curve for the matched image should lie on the reference's curve rather than the source's.

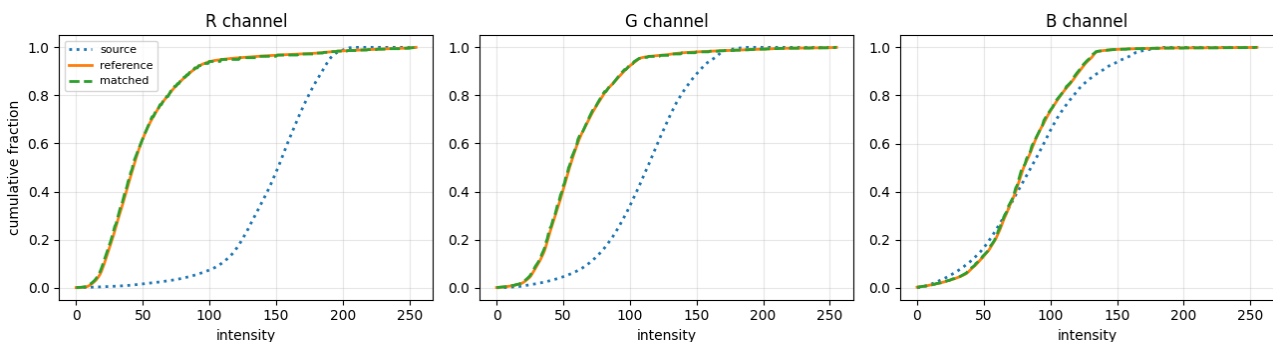


Figure 13 — cumulative histograms per channel for the source, reference and matched image.

Channel	Max CDF distance before	Max CDF distance after
R	0.868	0.019
G	0.645	0.017
B	0.091	0.015

The maximum distance between the cumulative distributions drops sharply in every channel. It does not reach zero, and for the same reason the equalised histogram was not perfectly flat: the images are discrete, so the target distribution can only be approximated.

8. Observations and conclusion

8.1 How the three histogram operations differ

Operation	What it does to the histogram	Typical use
Contrast stretching	spreads the existing shape over the full range	display low-contrast image
Equalisation	reshapes it so every level holds about the same amount	maximising global contrast
CLAHE	equalises small tiles independently	local detail, uneven lighting
Matching	reshapes it to follow a reference image	making two images tonally consistent

8.2 Points that required care

Issue	What happens	How it was handled
cv2.imshow in Jupyter	blocking window, can freeze the kernel	display with Matplotlib instead
Threshold of 0	not a no-op; comparison is strictly greater	verified against the count of zero-valued pixels
equalize_hist output	returns float in 0–1, not uint8 in 0–255	read the histogram range from the data
match_histograms API	multichannel was removed	use channel_axis=-1 for colour images
Discrete images	equalisation and matching are approximate	verified on the cumulative histograms

8.3 Conclusion

Thresholding is the simplest segmentation there is, and its entire difficulty lies in choosing one number. No fixed value transfers between images; Otsu's method removes the guesswork where the histogram has two distinct peaks, and adaptive thresholding handles the case where uneven lighting makes any single global value impossible.

Histogram processing improves contrast by reshaping the distribution of intensities rather than the arrangement of pixels. Stretching preserves the shape and clips the tails, equalisation flattens the distribution, and matching imposes a reference distribution. All three leave the spatial content untouched — only the values change — and because digital images are discrete, the last two can only approximate their target.

9. Resources

Hands-On Image Processing with Python (course book)

Histogram equalisation — scikit-image

https://scikit-image.org/docs/stable/auto_examples/color_exposure/plot_equalize.html

Histogram matching — scikit-image

https://scikit-image.org/docs/0.24.x/auto_examples/color_exposure/plot_histogram_matching.html

OpenCV thresholding documentation

<https://docs.opencv.org/>